

系统设计入门

system-design-primer 小白指南 —— 从一台服务器到百万用户

学习对象: github.com/donnemartin/system-design-primer (~180k★)

产出日期 2026-10-01 · 配套实验: `exercise/system_evolution.py` (已实测)

疑问即目录

1. 1 "系统设计"到底是什么？和写代码有什么区别？
2. 2 可扩展性为什么比"跑得快"更重要？
3. 3 延迟和吞吐量是什么？为什么要背"延迟数"？
4. 4 一台服务器怎么一步步变成百万用户系统？
5. 5 负载均衡器凭什么能"分均匀"？反向代理是什么？
6. 6 CAP 理论"三选二"到底在选什么？
7. 7 "数据还没更新"是哪种一致性？
8. 8 SQL 还是 NoSQL？怎么选不翻车？
9. 9 缓存为什么被称为"数据库的生命线"？
10. 10 消息队列怎么让系统扛住流量洪峰？
11. 11 系统设计面试的四步法怎么用？
12. 12 这些概念和我用过的工具有什么关系？
13. 13 这个项目除了读，还能怎么用？

第一部分 · 疑问逐条解答

"系统设计"到底是什么？和写代码有什么区别？

01

系统设计 System Design = 在写代码之前，决定"系统由哪些零件组成、零件之间怎么连接、每个零件扛多少压力"的顶层规划。这个项目是 GitHub 上最热门的系统设计教程（约 18 万 star），有中文版，被广泛用于后端面试准备。

写代码解决"这个函数怎么实现对"；系统设计解决"这一千台机器怎么分工"。

故事：你开了家奶茶店（一台服务器）。生意好到排队排到马路上，你有两个选择：A) 把老板娘练成三头六臂——**纵向扩展**（换更强的机器，有上限）；B) 再雇 3 个店员 + 1 个领班分派客人——**水平扩展**（加机器，几乎无上限）。系统设计研究的主要是 B 这条路上的所有问题：领班怎么排班（负载均衡）、常客直接报口味不出示订单（缓存）、先拿号后做茶（异步）。

写代码视角（放大镜）

`def get_user(id): ...`

SQL: `SELECT * FROM users`

关心：正确、优雅、可读

系统设计视角（望远镜）

负载均衡

Web×200

缓存

消息队列

数据库

监控

关心：扛得住、不宕机、花钱少

图1：同一个系统的两个视角——放大镜管一行行代码，望远镜管一群机器怎么协作

可扩展性为什么比"跑得快"更重要?

02

性能 Performance：一个请求处理得多快（延迟低不高）。**可扩展性 Scalability**：压力变 10 倍时，加机器能不能让系统继续撑住。

一个性能好但不可扩展的系统，就像一辆跑得飞快但只能坐 1 人的跑车——用户一多立刻趴窝。

例子（来自实验）：我们实验里那台"2ms 处理一个请求"的服务器，容量 500 请求/秒。流量 400/s 时延迟 10ms（健康），600/s 时**直接崩溃、请求被丢弃**——它不是慢慢变慢，是断崖式崩塌。

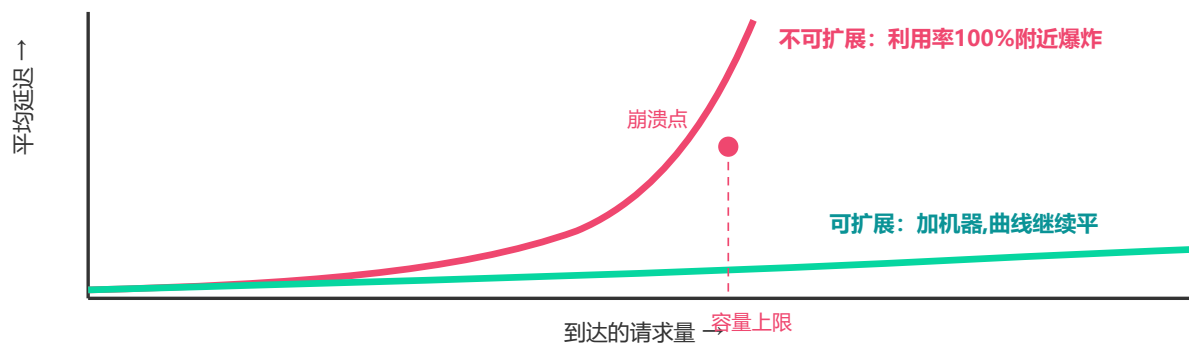


图2：红线（单机）到容量就爆炸；绿线（可水平扩展）加机器后延迟保持平缓

配套概念：**垂直扩展** 换更强的单机（简单但有物理上限、贵）；**水平扩展** 加更多普通机器（主流路线，需要本指南后面所有技术来配合）。

延迟和吞吐量是什么？为什么要背"延迟数"？

03

延迟 Latency：一个请求从发出到收到回复要多久（"快递几天到"）。**吞吐量 Throughput**：单位时间能处理多少请求（"快递站每天发多少件"）。两者往往要取舍。

教程附录那张"每个程序员都应该知道的延迟数"是全篇最值得背的表——它决定了你每个设计决定的性价比。



图3：延迟数直观化——这就是为什么"少一次磁盘/网络往返"是所有架构优化的第一性原理

一台服务器怎么一步步变成百万用户系统？

04

这是教程的主线故事（第"架构演进"部分）。每一步扩展都是被一个具体瓶颈逼出来的，不是为了炫技。

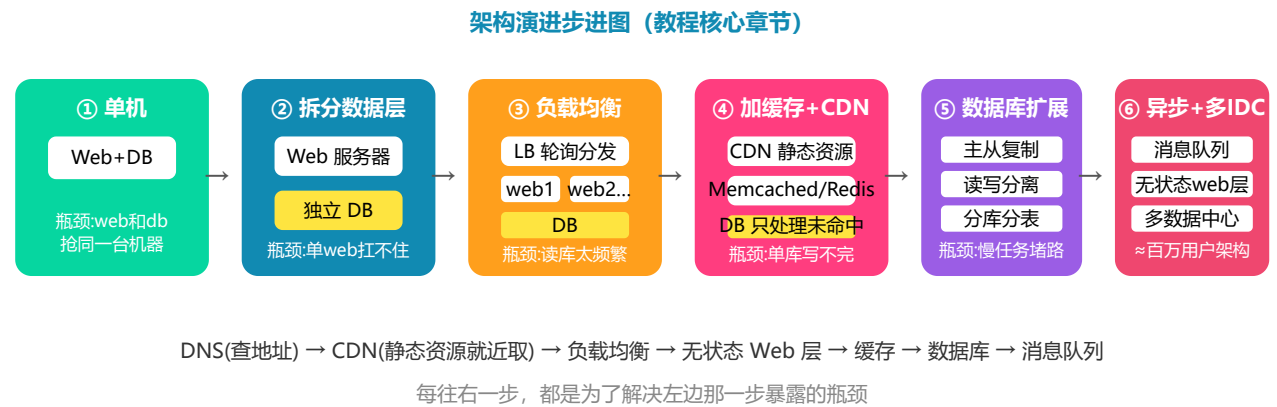


图4：六步演进——教程 README 用一整章+多张 SVG 讲这条路线，是全文的骨架

负载均衡器凭什么能"分均匀"? 反向代理是什么?

05

负载均衡器 Load Balancer：站在所有服务器前面的"领班"，把请求按策略分给后面某台机器。

反向代理 Reverse Proxy：同样站在服务器前面替它们收请求——负载均衡是它最重要的用途之一，两者概念高度重叠。

常见分发策略：轮询（挨个来）、加权轮询（能者多劳）、最少连接（谁闲给谁）、IP 哈希（同一用户固定去同一台，配合会话保持）。

它还兼职健康检查：某台机器挂了就把它踢出轮换，用户完全无感——这是"高可用"的第一块基石。



图5：负载均衡 = 领班派活 + 自动发现"病号"并隔离

注意区分：负载均衡器/反向代理站在**服务器这边**替服务器干活；教程里还提到**正向代理**站在**用户这边**替用户出面（比如公司统一出口）。方向相反，名字就像镜子一样对称。

CAP 理论"三选二"到底在选什么？

06

一致性 Consistency：所有节点同一时刻看到同一份数据。**可用性 Availability**：每个请求都能得到（非报错）响应。**分区容错性 Partition Tolerance**：机器之间网络断了，系统还能继续干活。

关键在于：网络分区不是"会不会发生"而是"何时发生"（网线、交换机、机房总会出事），所以 P 必选，真正的选择只有 C 和 A。

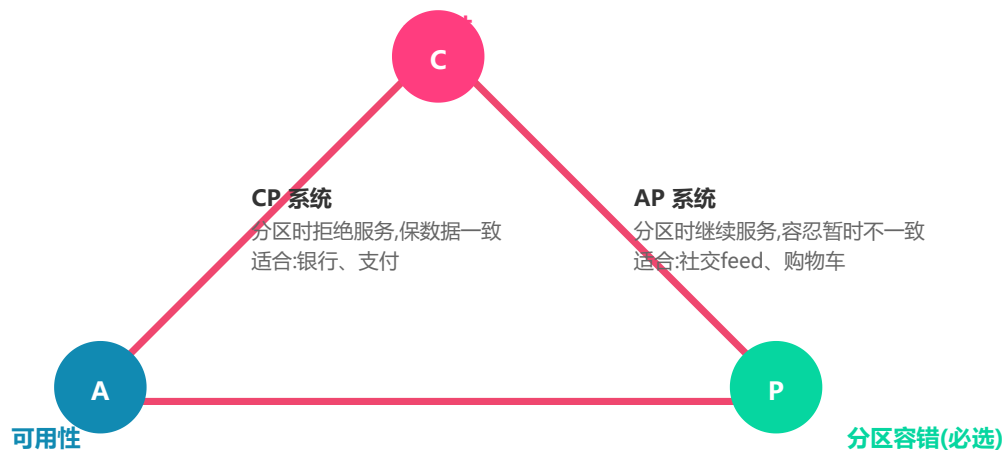


图6：CAP 三角——P 是现实强加的必选项，CP 和 AP 是两种产品性格

例子：转账时余额查询宁可不让你查（CP），也不能显示一个可能出错的旧数；朋友圈刷新时少一条新动态无所谓（AP），但不能整个刷不出来。教程原话：CP 适合"架构需要原子读写的场景"，AP 适合"继续运行但容忍脏读的场景"。

"数据还没更新"是哪种一致性?

07

强一致性：写入完成后，任何后续读取都能看到新值。**最终一致性**：写入后短暂读旧值，但保证"最终"读到新值。**弱一致性**：尽力而为，不保证什么时候能看到。

你改完微信头像、朋友那边过几秒才变——这就是最终一致性：它在"用户体验无感"和"系统开销小"之间做了漂亮的交换。

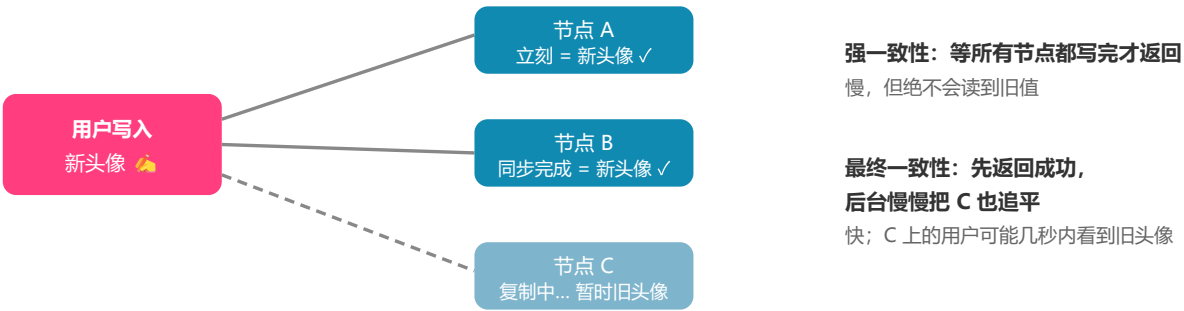


图7：数据多副本复制的两种节奏——"等齐了再说"vs"先答应后补课"

SQL 还是 NoSQL? 怎么选不翻车?

08

关系型数据库 RDBMS (MySQL/PostgreSQL)：表格 + 行列 + SQL 查询 + **事务 (ACID)**，天然强一致。**NoSQL**：键值 store (Redis)、文档库 (MongoDB)、宽表 (Cassandra)、图库 (Neo4j) 等，为海量数据和高吞吐而生，通常是最终一致。

维度	关系型 (RDBMS)	NoSQL
数据模型	表格，结构固定 (schema)	键值/文档/图，结构灵活
扩展方式	难水平扩展 (要分库分表)，通常向上扩展	天生为分布式水平扩展设计
一致性	强 (事务 ACID)	多为最终一致
适合	订单、账户、一切"错一点就出事"的数据	海量日志、feed 流、缓存、社交图谱
不适合	超大规模写、灵活多变的结构	需要复杂 join 和强事务的场景

教程给的默认路线：拿不准就用关系型——它久经考验；NoSQL 是当规模逼过来时才引入的针对性武器。

记忆钩子：RDBMS 像**规范的大酒店前台**（一切按登记簿来，查得慢但绝不出错）；NoSQL 像**灵活的便利店货架**（货随便摆、上货飞快，但偶尔标签还没贴好）。

缓存为什么被称为"数据库的生命线"?

09

缓存 Cache：把"贵的计算/查询结果"暂存在便宜快速的地方（通常是内存）。有效的原因是**局部性**：热门数据总被反复访问（80% 的请求打在 20% 的数据上）。

实验数据：10000/s 的读请求、DB 只扛 1000/s——命中率达到 95% 时打到 DB 的只剩 500/s，利用率从 1000% 降到 50%。没有缓存，数据库当场去世。



更新策略：缓存旁路(自己管) / 直写 / 回写(有丢失风险) / 到期驱逐(LRU)

缓存的代价：数据可能过期、要多管一层、要防"缓存雪崩"（缓存集体失效瞬间打穿 DB）

图9：读请求路径——缓存把数据库从"千分之十过载"救回"半负荷健康"

消息队列怎么让系统扛住流量洪峰?

10

消息队列 Message Queue：请求先入库排队，后台 worker 按自己的节奏消费。**异步**：把"慢活"从用户等待路径上摘掉。**背压 Back pressure**：队列太满时主动限流/拒绝，防止下游被压垮。

实验数据：洪峰 10 倍流量、持续 5 秒——同步架构 5 倍过载全线超时；异步架构 API 层秒级"已受理"，洪峰后约 20 秒排干积压。

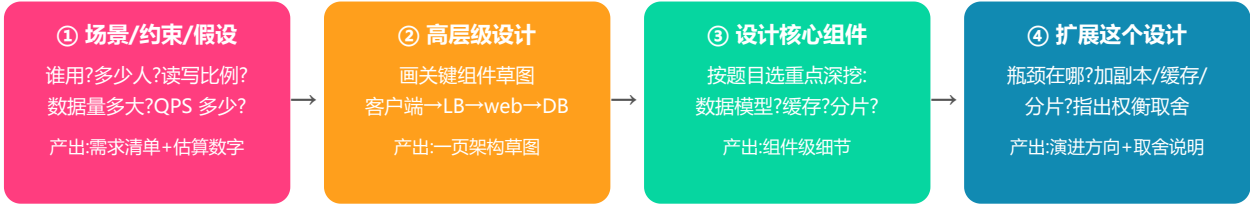


图10：消息队列 = 水库——上游随意洪峰，下游恒定泄洪；代价是"受理"不等于"完成"

系统设计面试的四步法怎么用？

11

教程的"如何处理一个系统设计面试题"给出万能框架。它的本质和知识库里的 [[构建流水线与Pass]] 同构：把一个吓人的开放题拆成顺序的四个阶段，每阶段有明确的产出物。



"估算法 back-of-envelope": 用 2 的次方表 + 延迟数, 30 秒心算出 QPS 和存储量

图11：四步法——每一步都有明确产出物，绝不会"不知道从哪下手"

估算法示例：做一个 Twitter 克隆, 3 亿月活 × 每天刷 2 次 × 每次读 100 条 → 读 QPS 量级 ≈ 10⁵~10⁶, 一眼就知道"必须有缓存和多级读扩展"。数字不需要精确, 量级对了, 设计方向就对。

这些概念和我用过的工具有什么关系？

12

已学过的东西	系统设计里的对应物	关联点
本地跑 Ollama 大模型	自托管服务 + 模型服务化	Ollama 就是把大模型封装成"本地 API 服务", 相当于你电脑上的微型 Web 服务层
LangChain / CrewAI 多智能体	微服务 + 服务发现	每个 Agent ≈ 一个独立服务; 它们的协作编排 ≈ 教程讲的"应用层拆分", 只是拆的粒度更细
[[构建流水线与Pass]]	请求处理管线	请求经过 LB→web→缓存→DB, 就是一条"各 Pass 顺序处理"的流水线
[[质检Gate与自我纠错循环]]	健康检查 + 限流熔断	负载均衡器定期"体检"后端 = Gate; 过载时拒绝请求 = 背压式自我保护
[[零依赖编程]] (本次实验)	——	实验脚本只用标准库模拟排队论, 同输入同输出, 方便验证

一句话打通：你以前学的"把大任务拆成流水线 + 设质检站", 在服务器世界里原样重演——只不过拆的对象从"代码步骤"变成了"整台机器"。

这个项目除了读，还能怎么用？

- **面试速查手册**：主题索引按概念组织（负载均衡、CAP、缓存...每个都有“不利之处/缺陷”小节——这是它比一般博客靠谱的地方，讲每个技术都先讲代价）。
- **8 道真题 + 完整解答**（`repo/solutions/system_design/`）：Pastebin、Twitter 时间线、网页爬虫、Mint、社交网络数据结构、搜索键值库、Amazon 销售排名、AWS 百万用户系统，每题含草图+估算步骤。
- **抽认卡 + 互动编程挑战**：README 顶部链接到 Anki 卡组 and LeetCode 式题库，方便间隔复习。
- **真实架构参考**：附录收集了 Netflix、Twitter 等公司真实架构和工程博客，面试前扫一遍很有故事可讲。
- **多语言/多格式**：有简中/繁中/日文翻译，可生成 epub 离线阅读（`generate-epub.sh`）。

第二部分 · 这个项目给我什么

能力清单：① 一套完整的后端架构词汇（DNS/CDN/LB/缓存/队列/分片/一致性...）；② 一个可复用的设计方法论（四步法+估算法）；③ 8 道可模仿的完整设计案例；④ 面试/评审时的“共同语言”。

能干什么（结合你的水平）：

- 给 AI Agent 应用做**架构评审**：你的 LangChain/CrewAI 项目如果上线给别人用，用户量上来后遇到的每个坑，教程里都有对应章节。
- 读技术文章/架构博客时**不再迷路**——那些“水平扩展、读写分离、最终一致”的词全部有了准确定义。
- 后端岗位**系统设计面试**的直接备考材料（本项目最初就是为此而写）。
- 设计自己的 side project 时，用四步法先画架构图再动手，避免“先写代码后补架构”。

第三部分 · 动手实验：亲眼看见系统“演变”

实验材料：`exercise/system_evolution.py`（零依赖，只用 Python 标准库，确定性输出）。它用一个简化的排队模型（M/M/1 近似： $\text{延迟} = \text{服务时间} \div (1 - \text{利用率})$ ），模拟“单机 → 负载均衡 → 缓存 → 异步”四个阶段。**所有命令和输出都已在 Windows + Python 3 实际验证。**

准备工作：打开 PowerShell，确保 UTF-8（避免中文乱码）：`setx PYTHONUTF8 1`（设置一次即可，重开终端生效）。

步骤 1 • 看单机如何“断崖式崩塌”

操作：进入实验目录并运行第一阶段：

```
cd F:\reminder\system_design_primer\exercise
python system_evolution.py single
```

目的：观察一台服务器（容量 500 请求/秒）在不同流量下的延迟。**实现了什么：**用排队公式算出各流量档位的平均延迟。

✅ **预期结果：**打印 5 行表格。流量 400/s 时延迟 10ms、状态“危险”；600/s 时显示 **OVERLOAD 崩溃(请求被丢弃)**。重点看延迟从 4ms→10ms→20ms→崩溃的**非线性**加速。

步骤 2 • 负载均衡：4 台机器把过载点右移 4 倍

操作：

```
python system_evolution.py lb
```

目的：验证“水平扩展=复制机器+均衡分发”。**实现了什么：**同样的排队模型，但容量变成 $4 \times 500 = 2000/s$ 。

✅ **预期结果：**同样形状的表格，但过载点从 ~500/s 右移到 ~2000/s；2500/s 才崩溃。对比步骤 1：**服务器数量与容量严格成正比**——这就是水平扩展的本质。

步骤 3 • 缓存：命中率救活数据库

操作：

```
python system_evolution.py cache
```

目的：观察缓存命中率对数据库压力的放大式缓解。**实现了什么：**10000/s 读请求 × 不同命中率 → 计算真正打到 DB 的流量。

✅ **预期结果：**命中率 0% 时 DB 利用率 **1000% (崩溃)**；命中率 95% 时利用率降到 **50% (健康)**。体会“命中率的 5 个百分点 = 数据库的一半寿命”这种杠杆效应。

步骤 4 • 异步：洪峰来了不崩，排队慢慢消化

操作：

```
python system_evolution.py burst
```

目的：对比同步 vs 异步+消息队列在 10 倍洪峰下的表现。

✅ **预期结果：**同步架构输出 **5 倍过载，全部请求超时**；异步架构输出 API 容量 20000/s 不崩、洪峰后约 **20 秒排干积压**。理解"用户拿到的是『已受理』而不是『已办完』"这一关键取舍。

步骤 5 • 做你自己的实验：改参数看变化

操作：用任意编辑器打开 `system_evolution.py`，找到 `REQ_MS = 2.0` 改成 `4.0`（服务器变慢一倍），保存后再跑：

```
python system_evolution.py single
python system_evolution.py lb
```

目的：验证你真的理解了模型： $\text{容量} = 1000 \div \text{REQ_MS}$ 。实现了什么：单机容量从 500/s 掉到 250/s。

✅ **预期结果：**步骤 1 表格里 250/s 就进入"危险"区、500/s 就崩溃；步骤 2 的 4 台容量变成 1000/s。改对了，说明排队模型你已经拿下了。

第四部分 • 坑与结论

- **坑：数字与结论不一致。**第一版实验脚本的模拟参数和文字结论对不上（表格里全部 OVERLOAD，结论却说"健康"）。修正方法：先想清楚每个阶段"要讲什么故事"，再反推参数。教训：**模拟器必须为叙事服务，且每个数字都要能被公式解释。**
- **坑：利用率 80% 是隐形红线。**从 2ms→4ms→20ms，利用率从 50%→80%→90%，延迟翻倍再翻倍。生产系统一般在 70~80% 就报警。
- **结论 1：**系统设计的所有技术（LB/缓存/队列/分片）都在回答同一个问题——**下一个瓶颈是什么，怎么买 10 倍余量。**
- **结论 2：**每个技术都有代价（缓存=数据可能过期；异步=受理≠完成；CP=拒绝服务）。**系统设计能力 = 权衡取舍的能力**，这也是面试真正考察的东西。
- **结论 3：**教程的正确打开顺序 = 先读"架构演进"主线（README 前部）→ 再按主题索引补概念 → 最后用 4 步法做 1~2 道真题（solutions/ 里有完整答案对照）。

